



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



COURIER PARCEL TRACKING SYSTEM

Network-Based Parcel Status Tracking using Python Socket Programming,
Cisco Packet Tracer and Wireshark

Capstone Project – Review 2

Team Members: U.Lakshikhaashri - (Hub Network Design)

S.Aarathana - (Parcel Status Update Program)

Ayesha Siddiqua. A - (Wireshark Verification)

Guided By: Jai Sharma

Table of Contents

1. Abstract
2. Introduction
3. Problem Identification and Analysis
4. Solution Design and Implementation
5. Software / Hardware Functionality
6. Modules Implementation – Technical Complexity
7. Result and Comparison (Module wise)
8. Limitations and Future Scope
9. Use of Ethics and Engineering Standards
10. Conference / Paper Presentation / Hackathon
11. Conclusion
12. References

Abstract

- The Courier Parcel Tracking System simulates the way courier companies such as DTDC, Blue Dart, Delhivery, DHL, FedEx and India Post track a parcel from the sender to the receiver.
- Every parcel is assigned a unique tracking number. As the parcel moves through courier hubs, each hub reports the new status to a central tracking server.
- A LAN of three courier hubs and one tracking server is designed in Cisco Packet Tracer with the 192.168.1.0/24 address space.
- Python TCP socket programs act as the hub clients and the central server; the server maintains an ordered status history for every parcel ID.
- Wireshark captures traffic on TCP port 5000 to verify that all three status updates reach the server reliably and in the correct order.
- Outcome: a working demonstration of client–server communication, centralised state maintenance and packet-level verification of a real courier workflow.

Introduction

- Courier tracking is one of the most widely used client–server applications in daily life.
- Each hub that handles the parcel becomes a client that pushes a status update to a central server; the customer only reads the consolidated history.
- This project recreates that process with three tools:
 - Cisco Packet Tracer – design of the hub-to-server network topology and IP plan.
 - Python 3 socket programming – real TCP communication between hubs and the server.
 - Wireshark – packet-level analysis to prove correct and ordered delivery.

Real-Time Scenario – Tracking ID : P101



Problem Identification and Analysis

Problem Statement

- Courier companies handle thousands of parcels every day. Without a centralised tracking system the movement of a parcel cannot be observed by anyone.

Issues identified

- Customers cannot know the current location or stage of the parcel.
- The company cannot monitor parcel movement between regional hubs.
- Lost or stuck parcels are difficult to identify because no history exists.
- Duplicate, missing or out-of-order status updates corrupt the record.

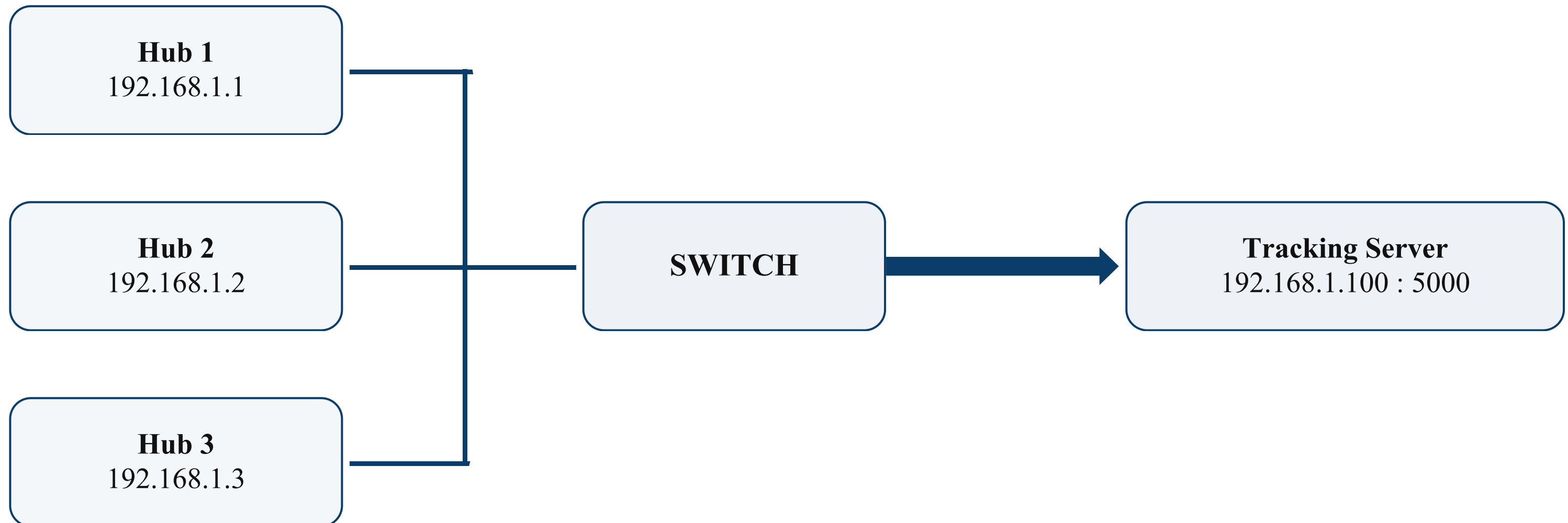
Analysis / Requirement derived

- A single authoritative server must own the parcel history, hubs must only append updates, and the transport must be connection-oriented (TCP) so that no update is lost or reordered.

Solution Design and Implementation

System Architecture

- Three hub PCs and one tracking server connected through a single switch on 192.168.1.0/24.
- Hubs act as TCP clients; the server listens on 0.0.0.0:5000 and stores history in memory keyed by parcel ID.



Software / Hardware Functionality

Software Used

Software	Purpose
Cisco Packet Tracer	Network design & IP addressing
Python 3	Client–server socket programs
Wireshark	Packet capture and analysis
VS Code	Code development and testing

Hardware Requirements

Component	Specification
Computer / Laptop	Any x86-64 system
RAM	4 GB or above
Operating System	Windows / Linux
Network	LAN; Internet only for setup

IP Addressing Plan (Subnet Mask 255.255.255.0)

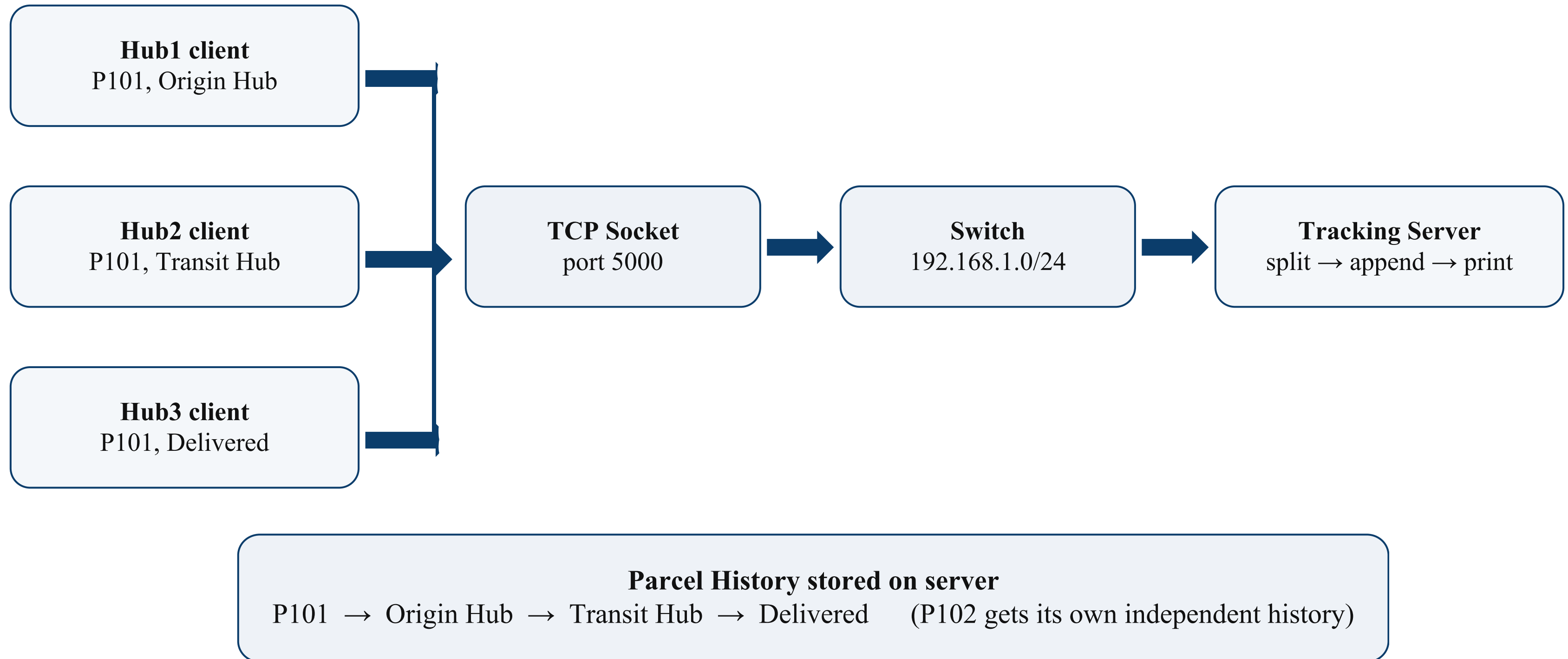
Device	IP Address	Role	Connectivity Test
Hub1	192.168.1.1	Origin hub client	ping 192.168.1.100
Hub2	192.168.1.2	Transit hub client	ping 192.168.1.100
Hub3	192.168.1.3	Destination hub client	ping 192.168.1.100
Server	192.168.1.100	Tracking server (TCP 5000)	netstat / listening

Modules Implementation – Server Algorithm (Module 2)

Components : `server.py`, `hub1.py`, `hub2.py`, `hub3.py`

- Step 1 – Server starts: `python server.py` → "Courier Tracking Server Started..."
- Step 2 – Create TCP socket (AF_INET, SOCK_STREAM) for reliable, ordered delivery.
- Step 3 – Bind to 0.0.0.0 : 5000 so that any hub in the LAN can connect.
- Step 4 – `listen()` → "Waiting for Hub Connections..."
- Step 5 – `accept()` a hub connection (Hub1 first).
- Step 6 – Receive message in the format `ParcelID,Status` e.g. `P101,Origin Hub`
- Step 7 – Split on ',' → `parcel ID = P101`, `status = Origin Hub`.
- Step 8 – Append status to `history[P101]`; create the list if the parcel is new.
- Step 9 – Hub2 sends `P101,Transit Hub` → appended after Origin Hub.
- Step 10 – Hub3 sends `P101,Delivered` → appended; tracking complete. Print history and loop back to `accept()`.

Technical Complexity – Data Flow / Flow Chart



Result and Comparison (Module wise)

Module	Student	Work Done	Result Obtained
Module 1 Hub Network Design	Student 1	1 server, 1 switch, 3 hub PCs in Packet Tracer; static IPs on 192.168.1.0/24	ping 192.168.1.100 successful from all three hubs – full connectivity
Module 2 Parcel Status Update	Student 2	server.py + hub1/2/3.py using TCP sockets on port 5000	Server printed history: Origin Hub → Transit Hub → Delivered for P101; multiple parcels tracked
Module 3 Wireshark Verification	Student 3	Capture on Ethernet/Wi-Fi with filter tcp.port == 5000	3 update packets captured in order (Origin, Transit, Delivered) with SYN/ACK handshake – no loss

Server Console Output

Courier Tracking Server Started...
Waiting for Hub Connections...
Parcel : P101 History : Origin Hub
Parcel : P101 History : Origin Hub, Transit Hub
Parcel : P101 History : Origin Hub, Transit Hub, Delivered

Comparison – Before vs After

Aspect	Manual	This System
Status visibility	None	Full history
Update order	Unreliable	Guaranteed by TCP
Verification	Not possible	Wireshark proof

Limitations and Future Scope

Limitations

- Works only within a local network (LAN).
- No database – history is lost when the server stops.
- No graphical user interface for customers.
- No user login or role-based access.
- Data is transmitted without encryption.
- Server handles hubs sequentially, not concurrently.

Future Scope

- MySQL database for permanent parcel history.
- Web application for customers to view live status.
- User authentication for hub operators.
- GPS-based real-time location tracking.
- SMS / e-mail notification on every status change.
- Mobile app and TLS-encrypted communication.

Use of Ethics and Engineering Standards

Engineering Standards Followed

- IEEE 802.3 Ethernet and IEEE 802.11 wireless for the physical/data-link layer of the hub LAN.
- TCP/IP suite (RFC 793 – TCP, RFC 791 – IPv4) for connection-oriented, ordered and reliable status delivery.
- IPv4 private addressing as per RFC 1918 (192.168.1.0/24) with a documented subnetting plan.
- Registered/user port range usage: application port 5000 kept outside the well-known port range.
- PEP 8 coding conventions and modular design for the Python client and server programs.

Professional Ethics Practised

- Only synthetic parcel IDs and statuses are used – no real customer data, addresses or phone numbers.
- Packet capture is performed on the team's own laboratory network with permission; no third-party traffic is inspected.
- Licensed / free educational software only (Cisco Packet Tracer academy edition, Wireshark and Python – open source).
- All references and sources are duly cited; the implementation is the team's original work.
- Limitations (no encryption, no persistence) are disclosed honestly rather than overstated.

Conference / Paper Presentation / Hackathon

Paper prepared from this project

- Title: "A Socket-Based Centralised Courier Parcel Tracking System with Packet-Level Verification".
- Contribution: an inexpensive, reproducible laboratory model that couples network design (Packet Tracer), TCP socket implementation and Wireshark validation of update ordering.

Submission / Participation plan

Event Type	Event / Venue	Status
National Conference	Conference on Computer Networks & Communication (department-level ICCNC)	Paper drafted – to be submitted
Paper Presentation	Intra-college technical symposium	Presented / demo shown
Hackathon	Smart India Hackathon – logistics tracking theme	Idea submitted for shortlisting

Deliverables: 6-page IEEE-format paper, network topology file, source code repository and Wireshark capture (.pcapng) as supporting evidence.
(Replace event names, dates and certificate details with the actual ones before the final university presentation.)

Conclusion

- The Courier Parcel Tracking System successfully demonstrates how a courier company tracks parcels using a client–server architecture.
- The Packet Tracer topology provides the communication network between the courier hubs and the central tracking server.
- The Python socket programs enable real-time status updates, and the server maintains a complete, ordered history for every parcel ID.
- Wireshark verification confirms that all updates are transmitted reliably and in the correct sequence on TCP port 5000.
- The project therefore combines core networking concepts with practical programming and reflects the basic workflow used in real courier tracking services.
- With a database, a web interface and encryption, the same design can be extended into a deployable production tracking service.

References

- A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed., Pearson, 2011.
- J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.
- W. R. Stevens, UNIX Network Programming, Vol. 1: The Sockets Networking API, 3rd ed., Addison-Wesley, 2004.
- RFC 793 – Transmission Control Protocol, IETF, 1981.
- RFC 1918 – Address Allocation for Private Internets, IETF, 1996.
- Python Software Foundation, "socket — Low-level networking interface", docs.python.org/3/library/socket.html.
- Wireshark Foundation, Wireshark User's Guide, www.wireshark.org/docs.
- Cisco Networking Academy, Cisco Packet Tracer Tutorials, www.netacad.com.
- Blue Dart / Delhivery / India Post public parcel tracking portals (reference for status workflow).



Engineer to Excel



Thank You